



KusionStack + KCL

打造云原生时代开发者的自服务平台

李大元、宗喆

蚂蚁集团





Content 目录

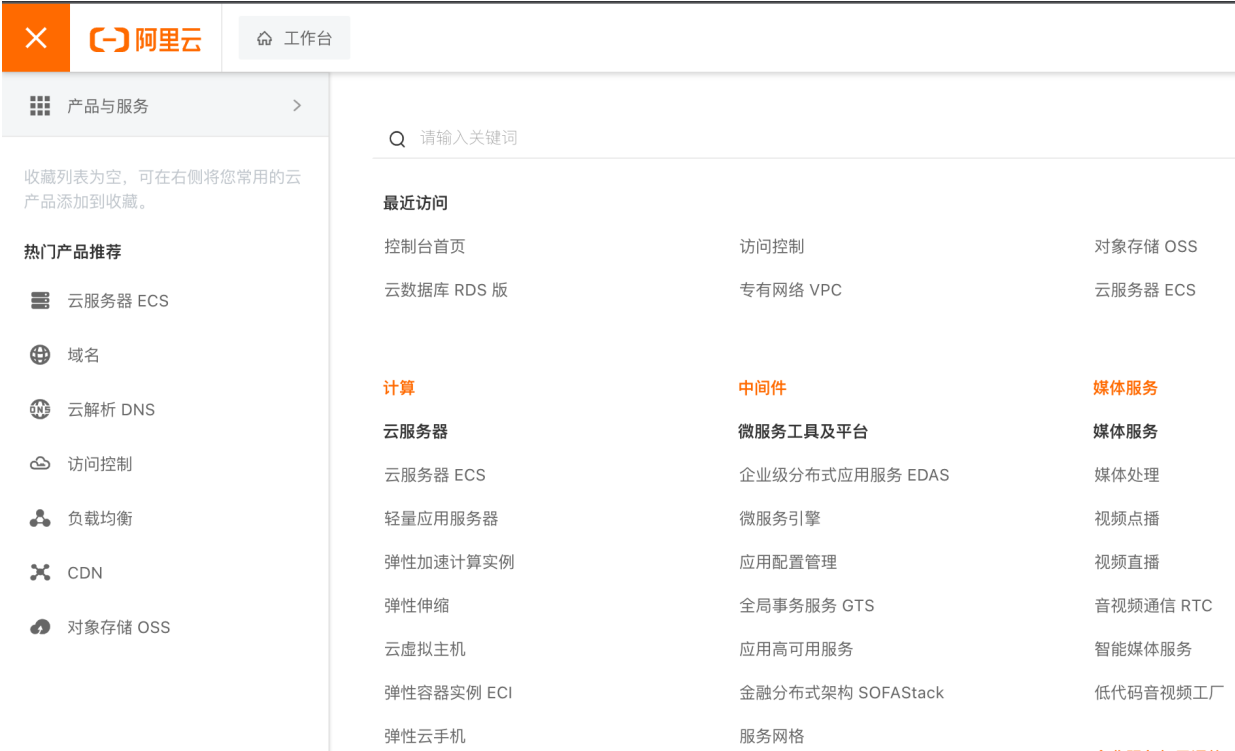
01 云原生时代的运维挑战

02 KusionStack—蚂蚁集团开源平台工程实践

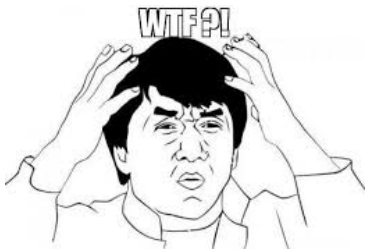
03 KCL

04 在蚂蚁和其他公司的实践案例

云原生时代的运维挑战

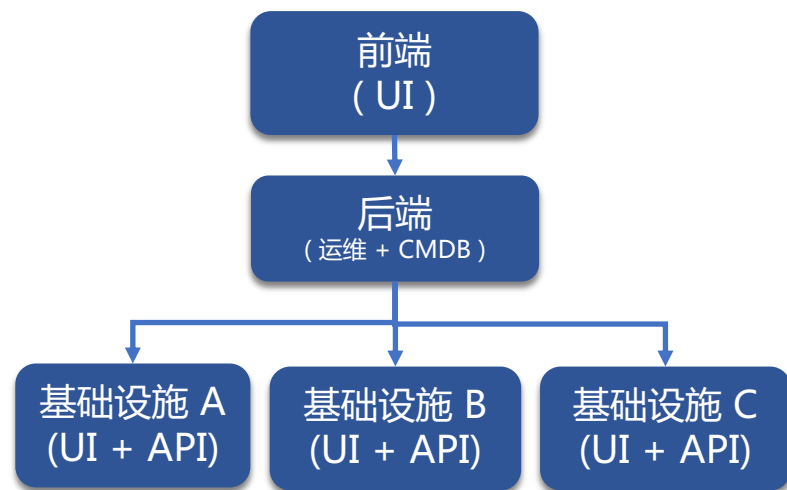


Source: [David Bell](#)



一次发布使用几个系统？找几个人？多久能上线？

传统 PaaS 的困境

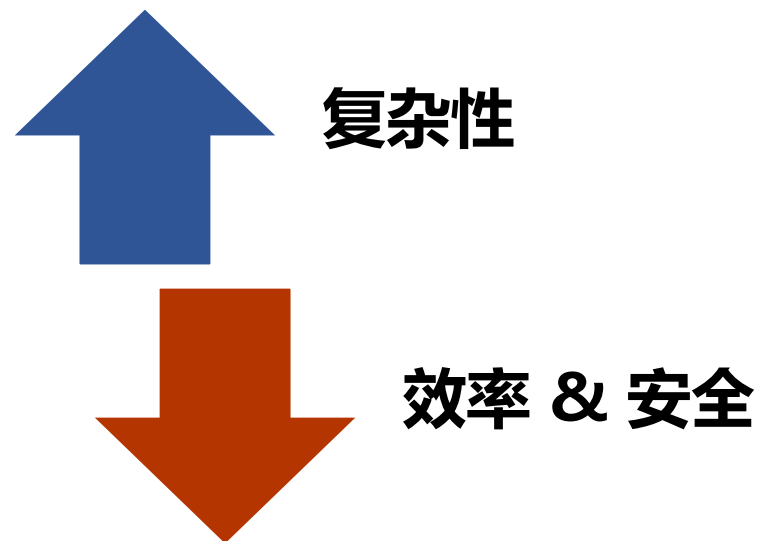


Developer

- 认知负担高
- 发布运维时间变长

Platform

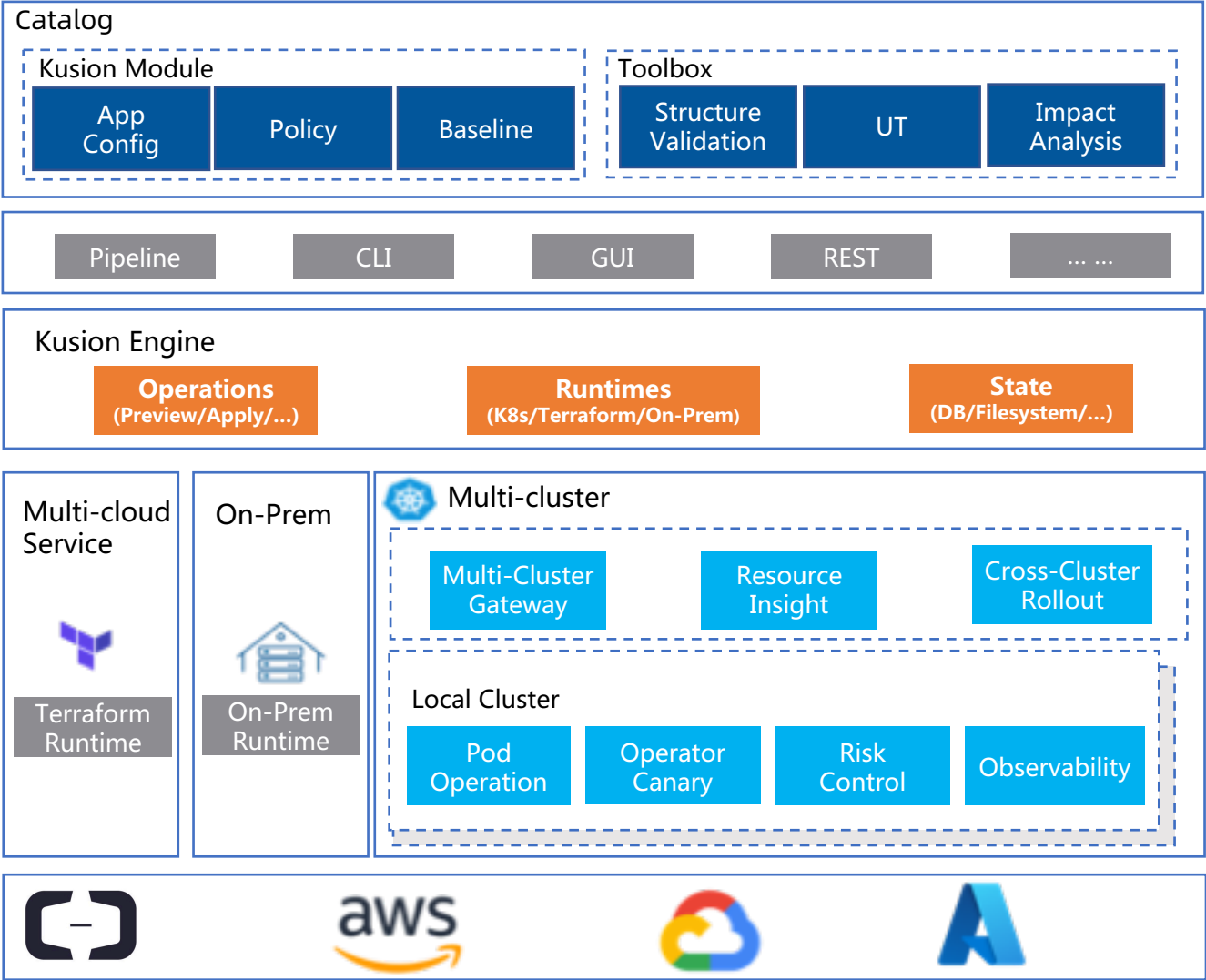
- 需求大幅增加，成为企业效能瓶颈
- 跨团队沟通成本大幅增加



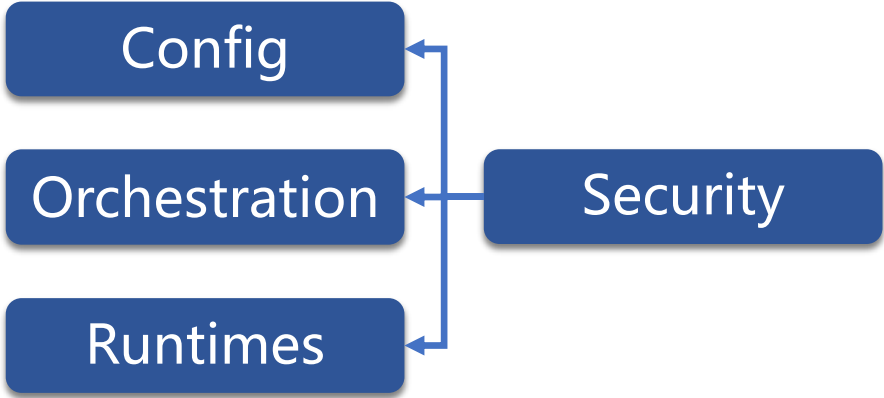
Security

- 多个独立基础设施平台，风险敞口大
- 声明式运维很强大也很危险

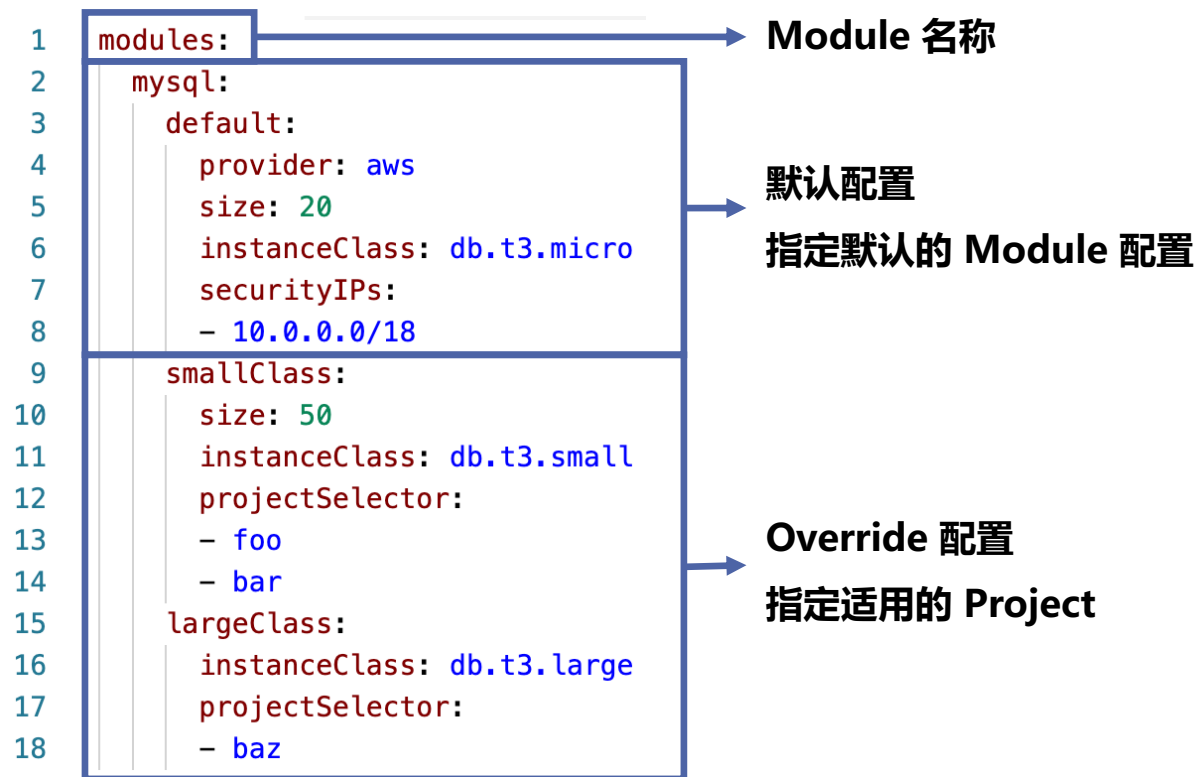
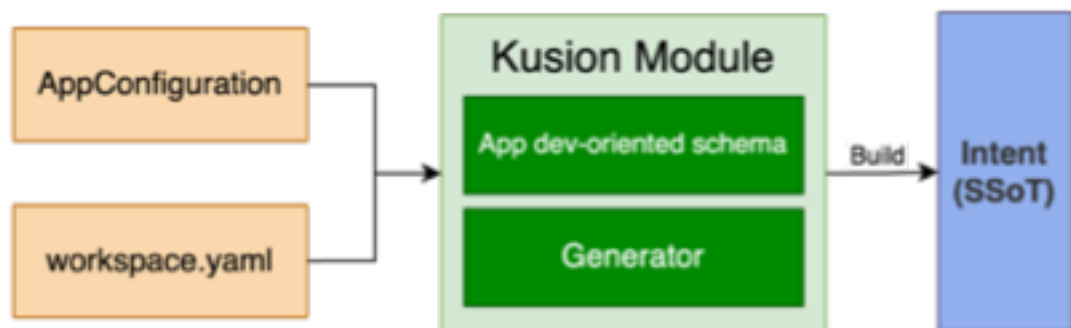
我们的实践— KusionStack 架构



助你更快、更安全的构建**自己的** Developer Platform



KusionStack—Module & Workspace



workspace.yaml

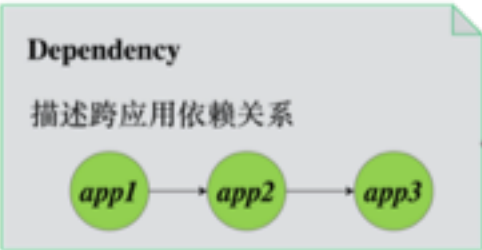
Platform 配置

- 可复用能力模块(资源、监控、中间件...), 实现**标准化**
- 差异化、复杂性留给平台, 降低研发**认知负担**
- 应用配置与平台配置解耦, 应用按需选择环境, 实现**一次编写, 多处部署**

KusionStack——AppConfiguration

App Dev 配置

- 使用可理解、易组合的 Module 实现**自服务**
- 一份配置涵盖应用交付**全生命周期**



```
1  helloworld: ac.AppConfiguration {
2    workload: wl.Service {
3      replicas: 2
4      containers: {
5        "nginx": c.Container {
6          image: "nginx:v1"
7          command: ["/bin/sh", "-c", "echo hi"]
8          env: {
9            "key": "value"
10         }
11         workingDir: "/tmp"
12         resources: {
13           "cpu": "2"
14           "memory": "4Gi"
15         }
16         readinessProbe: p.Probe {
17           probeHandler: p.Http {
18             url: "http://localhost:80"
19           }
20         }
21       }
22     }
23     ports: [
24       n.Port {
25         port: 80
26         targetPort: 8080
27         exposeInternet: True
28       }
29     ]
30   }
31   pipeline: {
32     "deploy": Deploy {
33       manualApprove: true
34       rollbackIfFailed: true
35     }
36   }
37   dependency: {
38     dependedApps: ["api-server"]
39   }
40 }
```

Components

描述完整交付应用所需的组件，包括工作负载 (workload) 和网络端口、数据库等配件 (accessory)

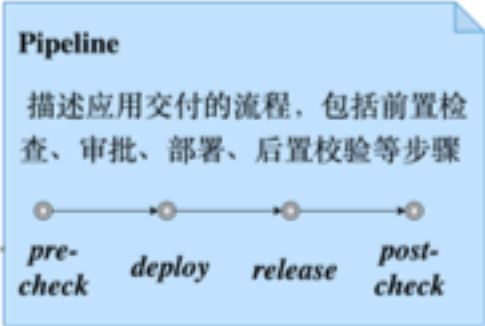
nginx

container

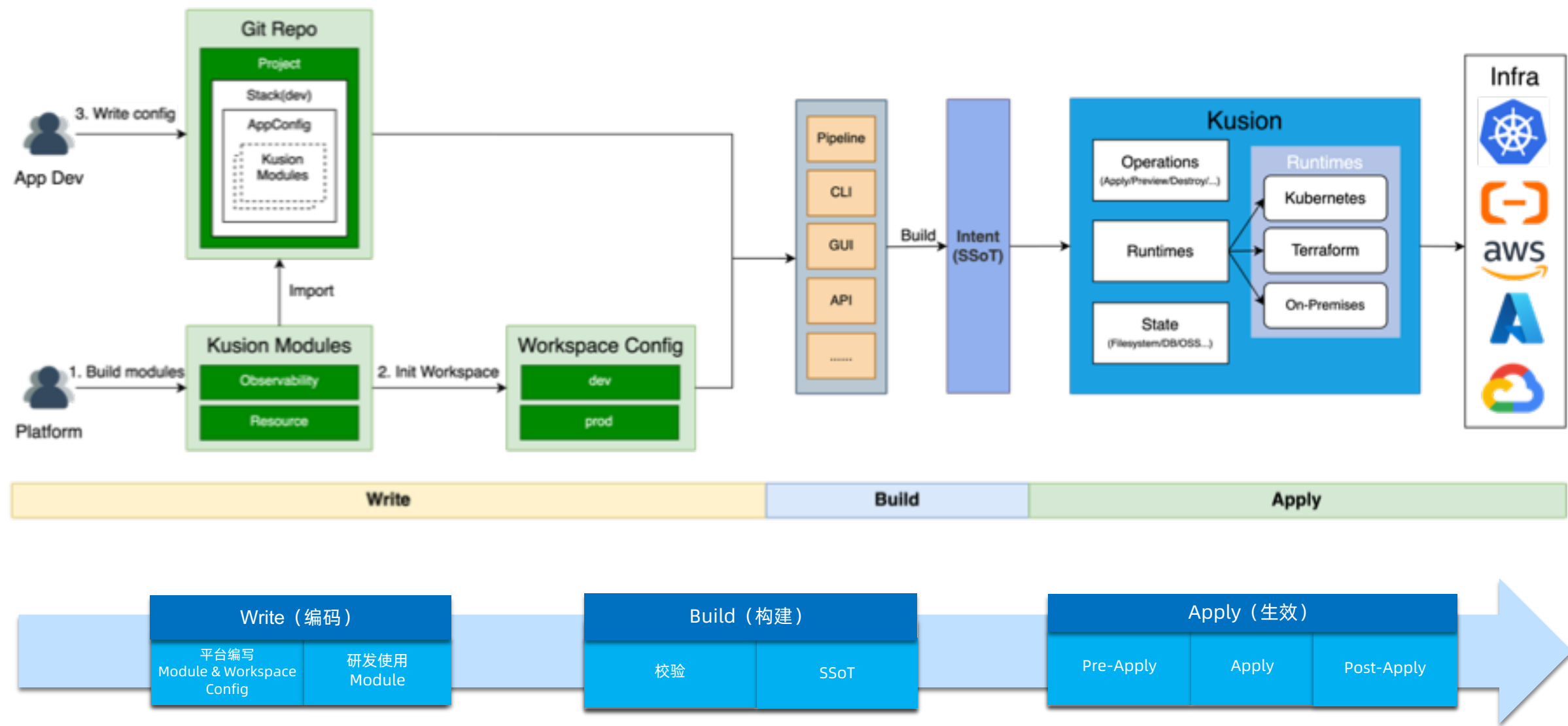
ports

database

accessory

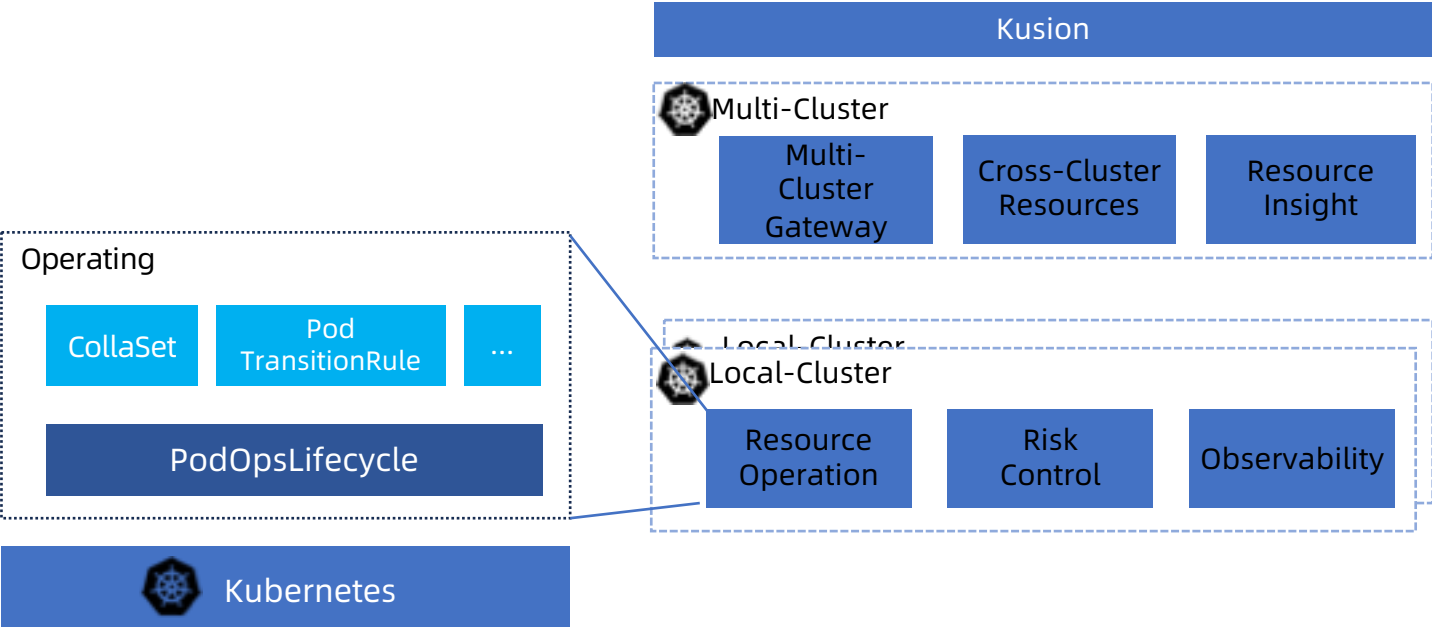


KusionStack—Workflow



云原生资源交付与运维

<https://github.com/KusionStack/operating>

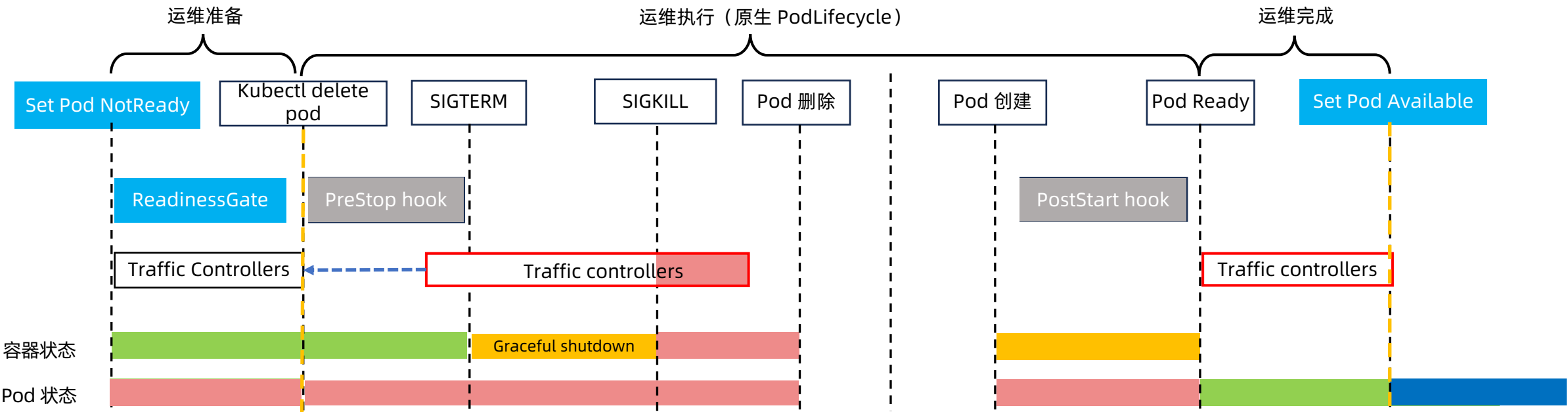


```
1 helloworld: ac.AppConfiguration {
2   workload: wl.Service {
3     replicas: 2
4     containers: {
5       "nginx": c.Container {
6         image: "nginx:v1"
7         command: ["/bin/sh", "-c", "echo hi"]
8         env: {
9           "key": "value"
10        }
11        workingDir: "/tmp"
12        resources: {
13          "cpu": "2"
14          "memory": "4Gi"
15        }
16        readinessProbe: p.Probe {
17          probeHandler: p.Http {
18            url: "http://localhost:80"
19          }
20        }
21      }
22    }
23  }
24 }
```

Pod 运维生命周期

原生 Pod 变更过程 K8s 提供的交互和管控能力有限

- Pod 粒度
- 可回滚
- 可扩展



Operator 稳定性管理

<https://github.com/KusionStack/controller-mesh>

单点问题:

高负荷

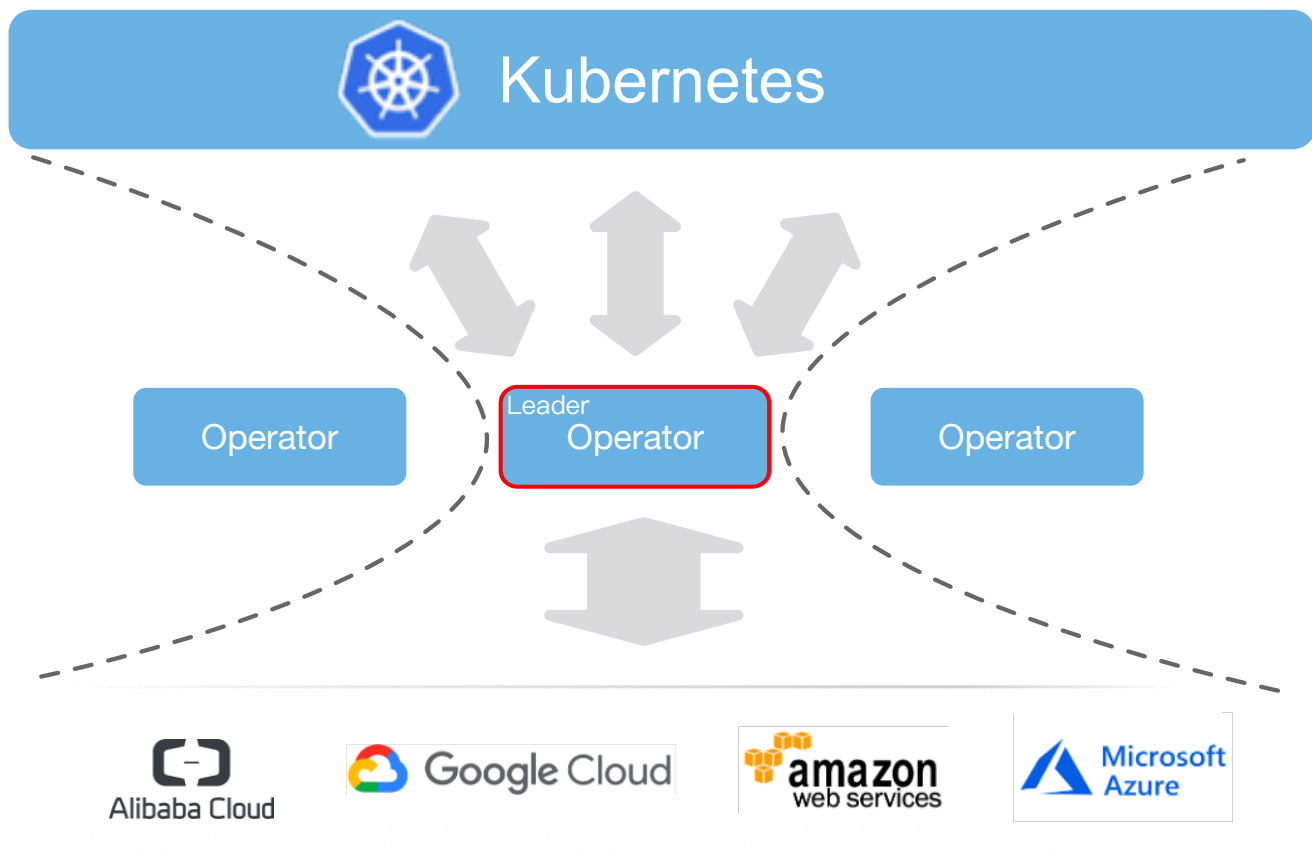
更多的 CPU & Memory
无法水平扩展
调和效率低
启动时间长

*List 700k pods -> 10min

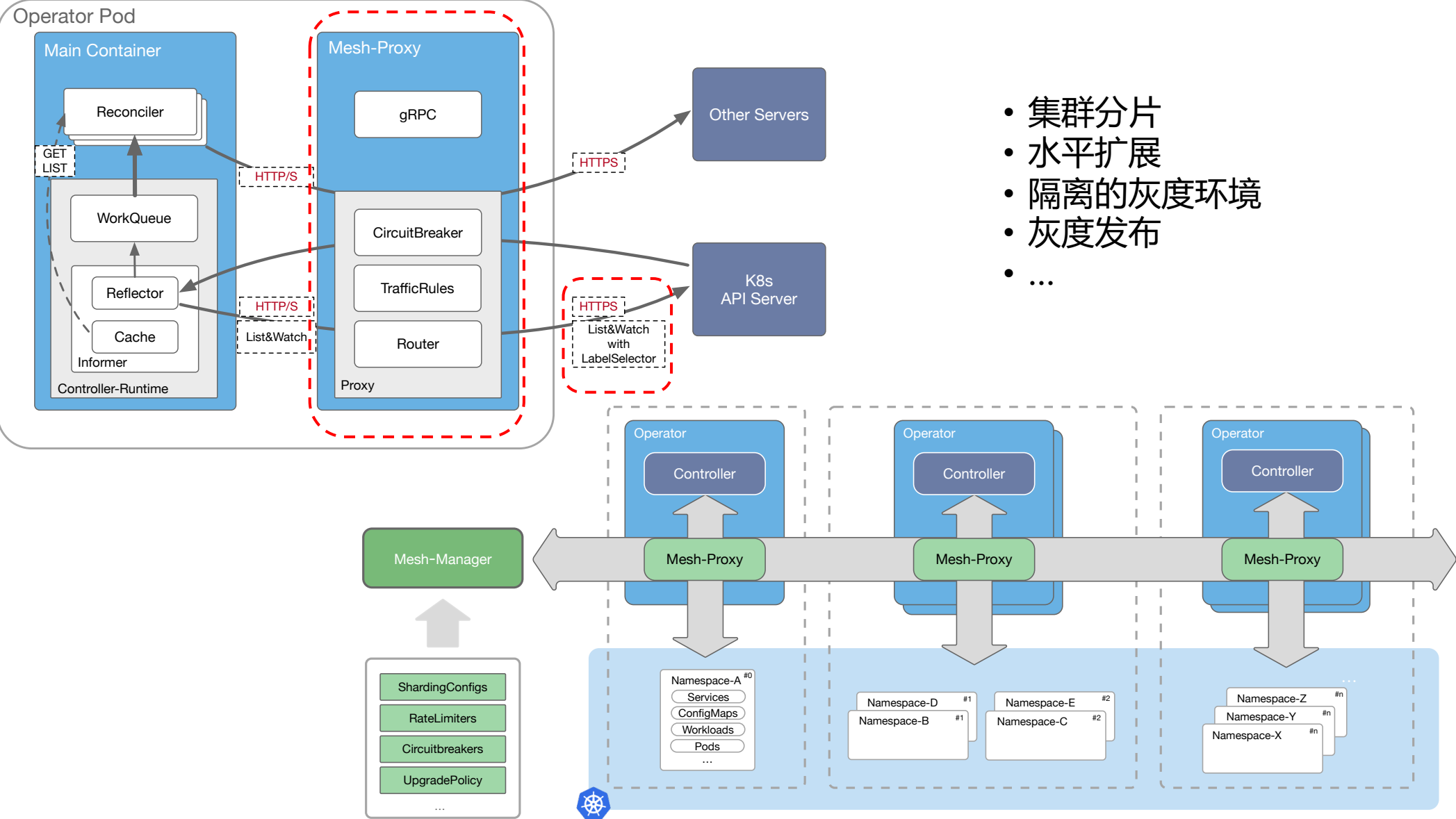
*Too old resource version

高风险

无法灰度
无法控制爆炸半径



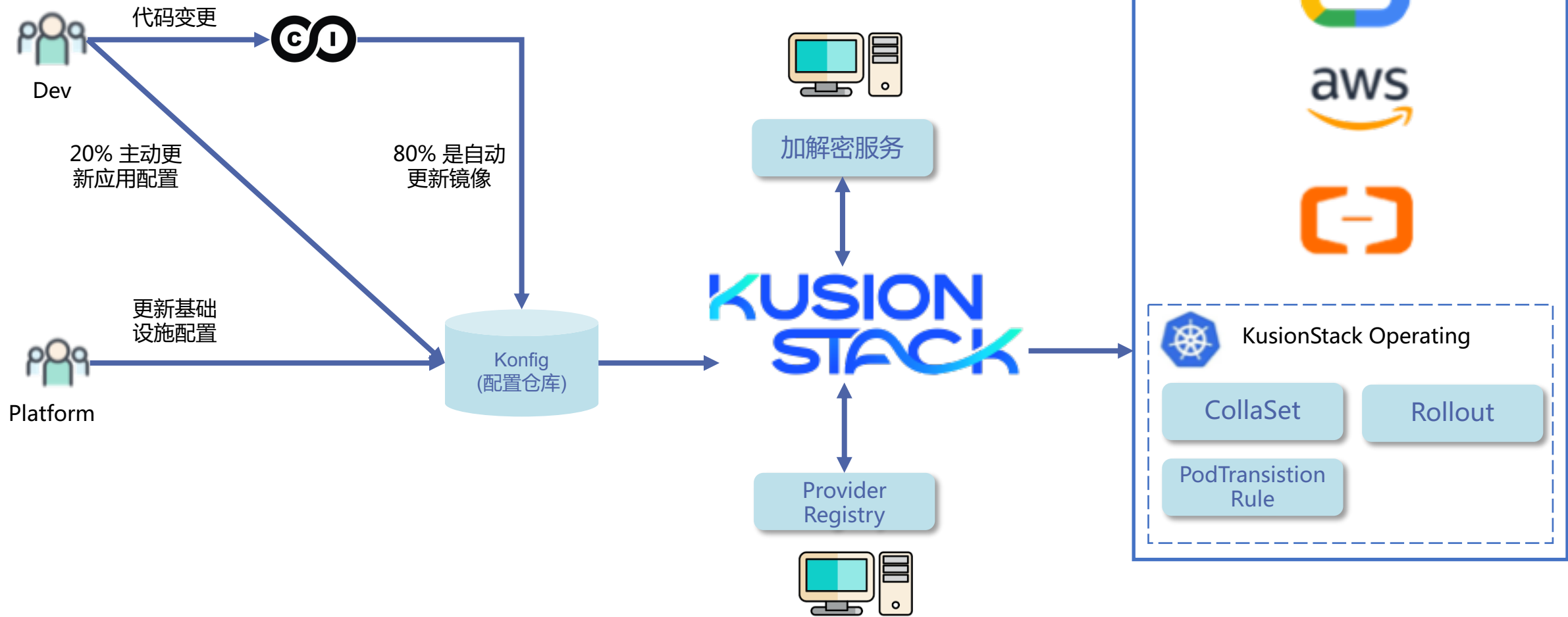
Operator 稳定性管理 -- ControllerMesh



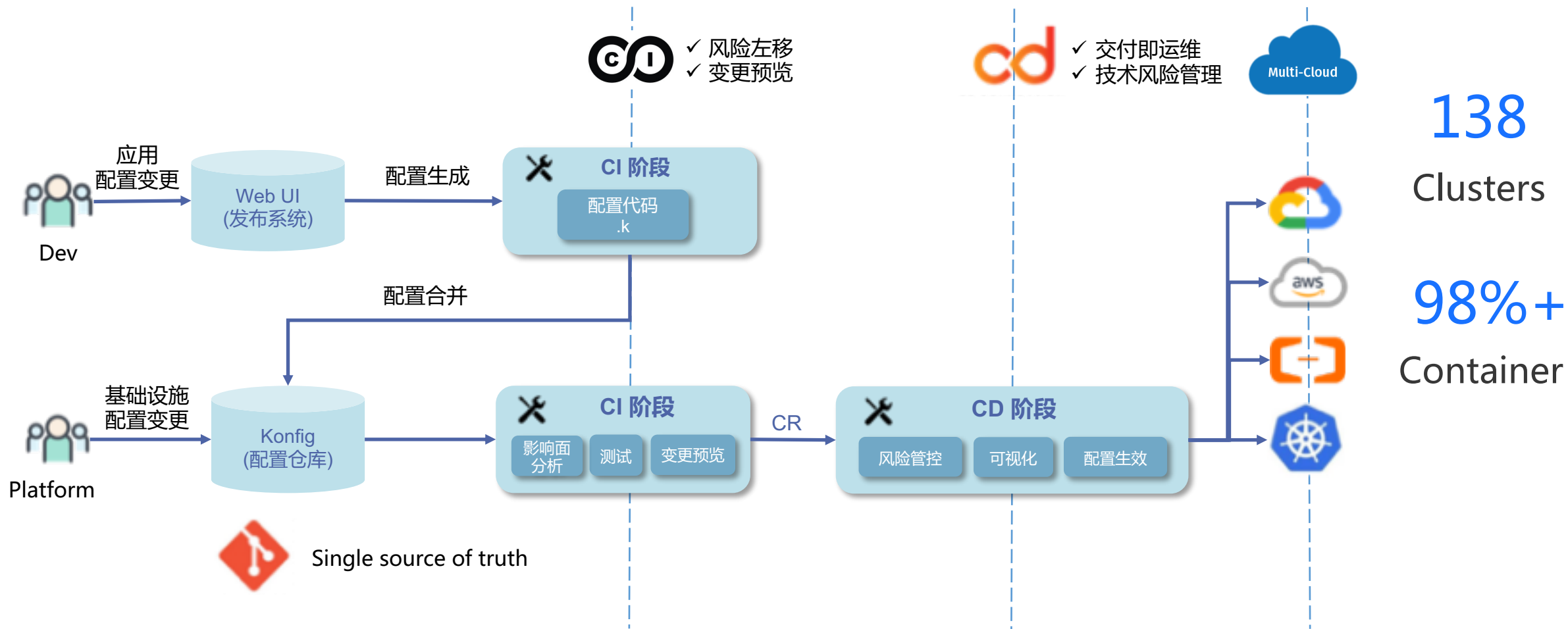
在蚂蚁和其他公司的实践



多云交付



在蚂蚁和其他公司的实践



Takeaways

1. 应用研发**认知负担**太高，生产力下降，阻碍企业创新
2. 平台成为企业效率的瓶颈，需要**新架构**释放生产力
3. 平台构建可复用 Module，研发**自服务**是可行的技术方案
4. 没有普适的 IDP，基于**开源**产品构建 IDP 的 ROI 更高

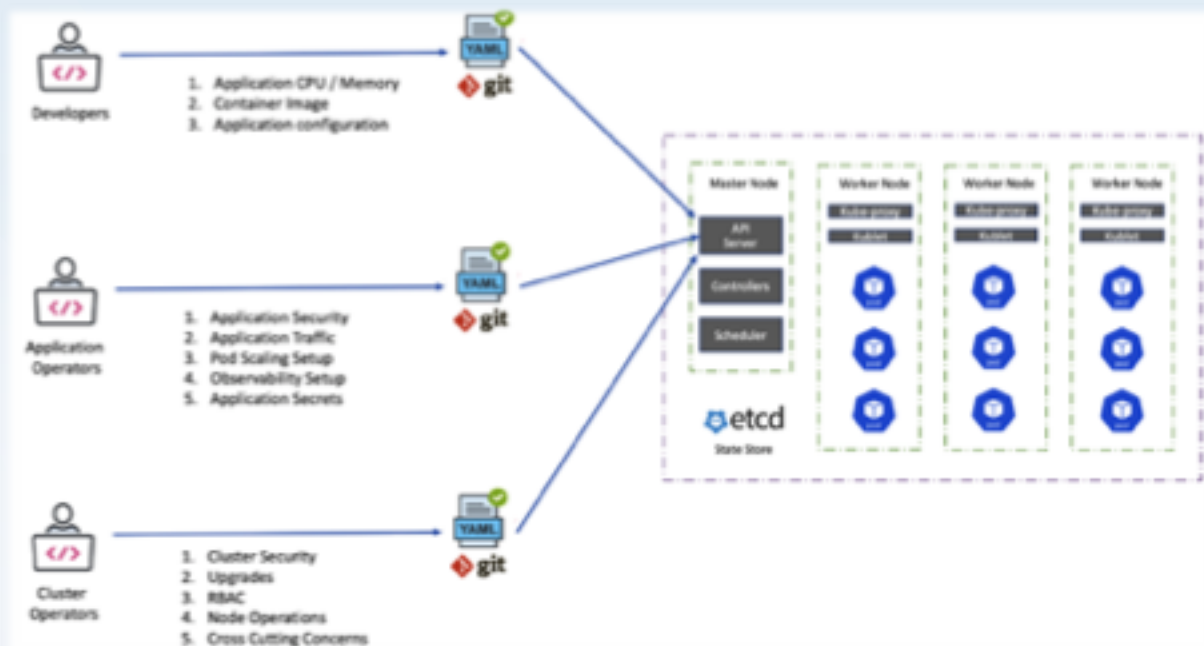


Part 02

KCL 云原生策略配置语言

云原生时代，基础设施即代码（IaC）是开发者体验的核心

基础设施代码化（IaC）已成为自动化和管理云资源的关键



需要一种减轻开发者认知负担和开发成本、提供高效动态配置管理，并且通过标准的配置测试与验证手段来保证可靠性的配置管理方式

认知负担

- 应用开发人员需要面对复杂的基础设施和平台概念
- 不像云基础设施配置有 Terraform 等 IaC 工具，针对 Kubernetes 平台缺乏轻量的配置组合和校验工具

静态配置

- YAML 膨胀，维度爆炸
- 跨团队配置协作负担和配置漂移

效率、可靠性低

- 缺乏标准的测试验证手段，大多是胶水代码或者脚本的拼盘
- 缺乏高效配置协同的工具，大多通过人肉拉群解决

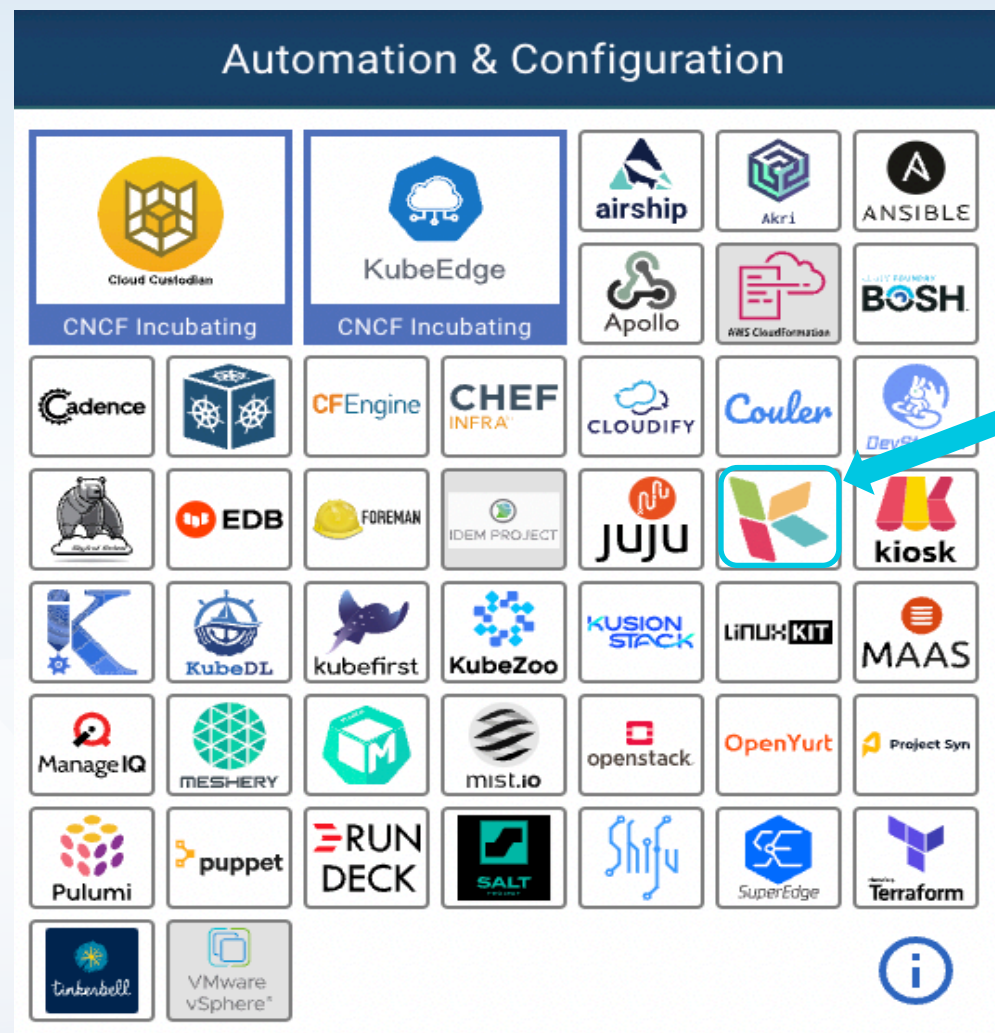
KCL 云原生策略配置语言

Mutation, Validation, Abstraction Production-Ready

KCL is an open-source constraint-based record & functional language mainly used in configuration and policy scenarios.

- ✓ **支持流程控制**：KCL 支持流程控制/lambda 表达式等基于现代编程语言元素提供的流程控制，提供了动态配置管理的能力。
- ✓ **支持配置测试与验证**：KCL 使用强类型系统，支持 assert, check, rule 等语言特性支持编写配置验证策略。
- ✓ **降低认知负担和开发成本**：通过 Schema 抽象数据结构，提供丰富的三方库，以应用为中心的模型屏蔽复杂的基础设施和平台概念。

面向云原生领域的专用配置策略语言
(2022.6 开源，2023.9 成为 CNCF 基金会托管的 Sandbox 项目)



KCL 语言特性

Mutation

流程控制/lambda 表达式

```
dataLoop7 = [i for i, _ in data]
```

```
x = lambda {  
    _e = 1  
    if True:  
        _e = 2  
    {  
        e: _e  
    }  
}
```

Validation

类型系统/约束定义/策略编写

```
x = "length"  
assert len(x) == 6 # True
```

```
schema Sample:  
    foo: str  
    bar: int  
    fooList: [str]  
  
    check:  
        bar > 0 #  
        bar < 100
```

Abstraction

Schema 结构体定义/三方库导入

```
schema Sample:  
    foo: str  
    bar: int variable is defined here,  
    fooList: [str]  
  
sample_inst = Sample(  
    foo: "foo",  
    bar: "1", expected int, got str(1)  
    fooList: ["foo", "bar"]  
)  
  
import k8s.api.core.v1 as k8core  
  
k8core.Pod {  
    metadata.name = "web-app"  
    spec.containers = [{  
        name = "main-container"  
        image = "nginx"  
        ports = [{containerPort = 80}]  
    }]  
}
```

KCL & KRM & 动态配置管理

• Mutation

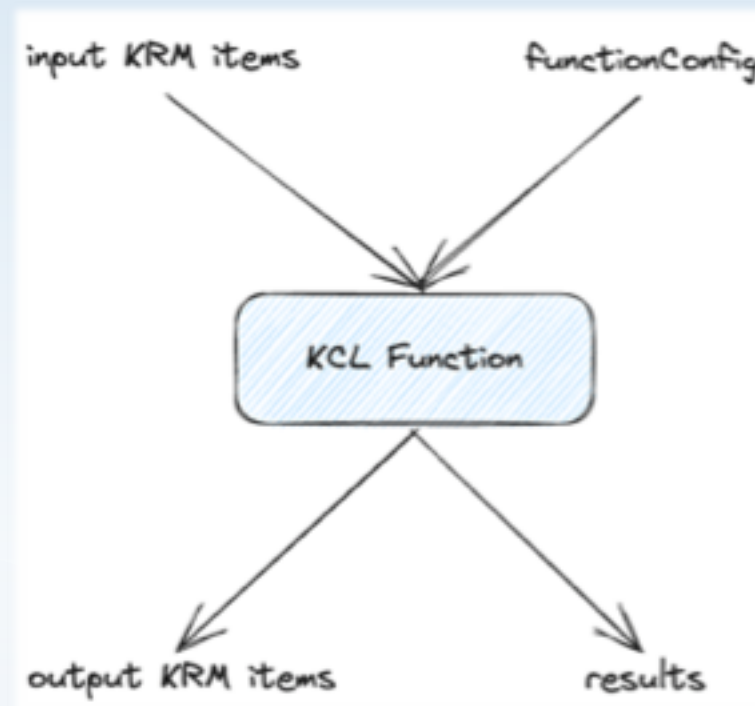
```
apiVersion: krm.kcl.dev/v1alpha1
kind: KCLRun
metadata:
  name: set-annotations
  metadata:
    annotations:
      krm.kcl.dev/version: 0.0.1
      krm.kcl.dev/type: mutation
      documentation: >-
        Add or change annotations
spec:
  params:
    toAdd: addValue
    source: oci://ghcr.io/kcl-lang/set-annotation
```

• Validation

```
apiVersion: krm.kcl.dev/v1alpha1
kind: KCLRun
metadata:
  name: https-only
  metadata:
    annotations:
      krm.kcl.dev/version: 0.0.1
      krm.kcl.dev/type: validation
      documentation: >-
        Requires Ingress resources to be HTTPS only. Ingress resources must
        include the `kubernetes.io/ingress.allow-http` annotation, set to `false`.
        By default a valid TLS {} configuration is required, this can be made
        optional by setting the `tlsOptional` parameter to `true`.
        More info: https://kubernetes.io/docs/concepts/services-networking/ingress/#tls
spec:
  source: oci://ghcr.io/kcl-lang/https-only
```

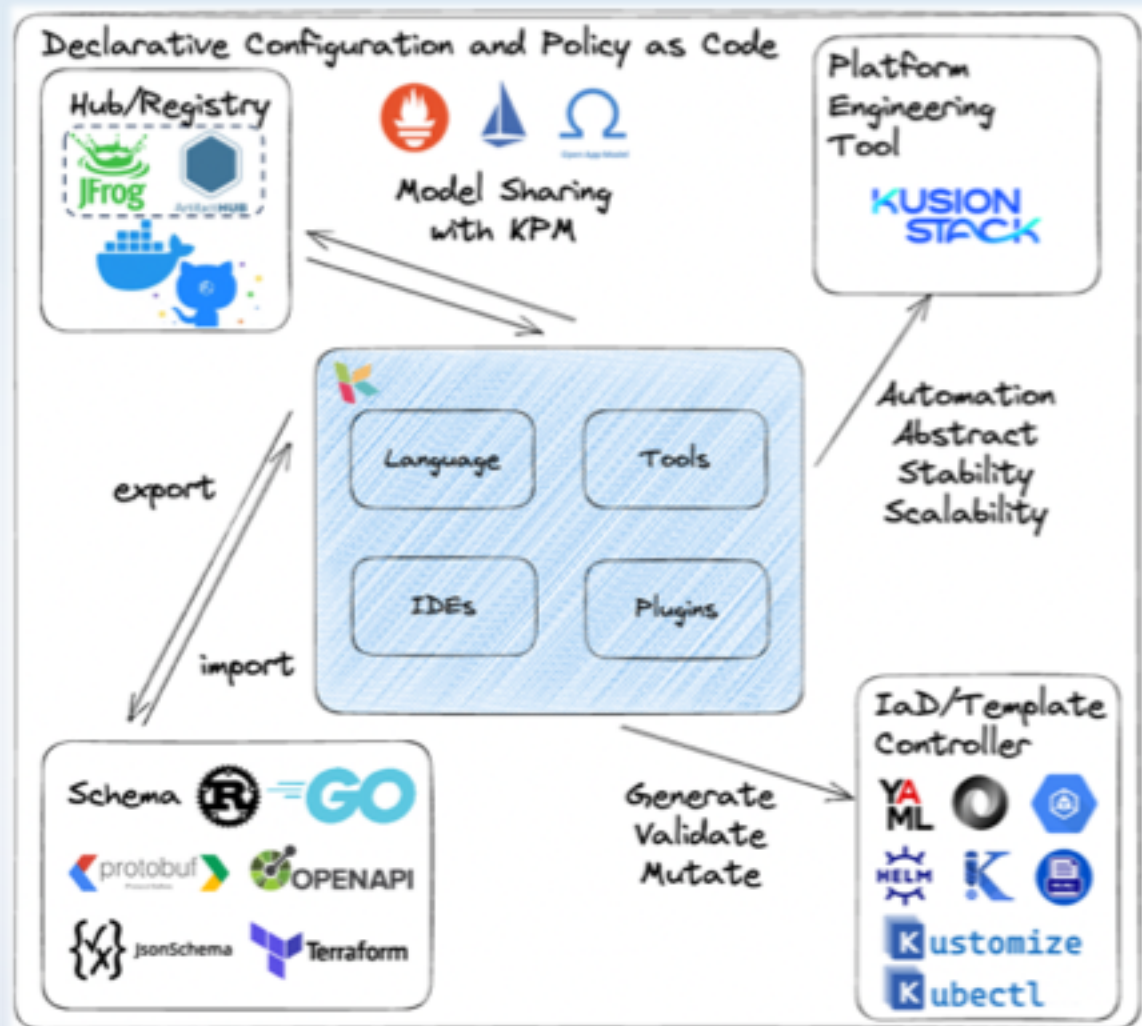
• Abstraction

```
apiVersion: krm.kcl.dev/v1alpha1
kind: KCLRun
metadata:
  name: web-service
  metadata:
    annotations:
      krm.kcl.dev/version: 0.0.1
      krm.kcl.dev/type: abstraction
      documentation: >-
        Web service application abstraction
spec:
  params:
    name: app
    containers:
      nginx:
        image: nginx
        ports:
          containerPort: 80
    labels:
      name: app
    source: oci://ghcr.io/kcl-lang/web-service
```



- 遵循统一的 KRM Function 规范
- 多种代码源支持: OCI, Git, Https, Filesystem...
- 可编程可扩展: 使用 KCL 语言简单编写模型

KCL 生态集成



Client

IaC/IaD

Helm/Kustomize/KPT/..
KCL Plugins

Runtime

Admission Request
Mutating/Validating
Webhook

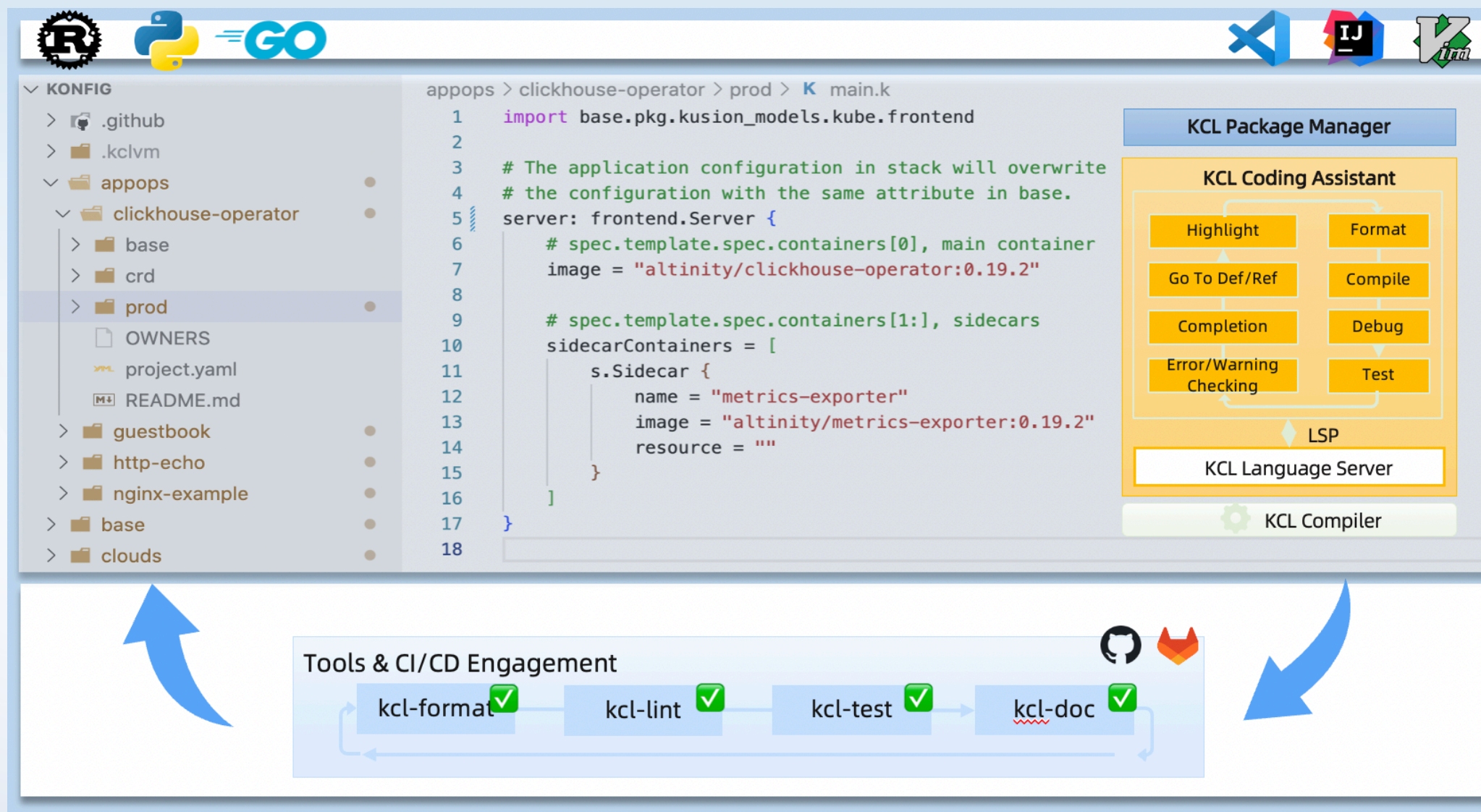
KCL Operator

SDKs



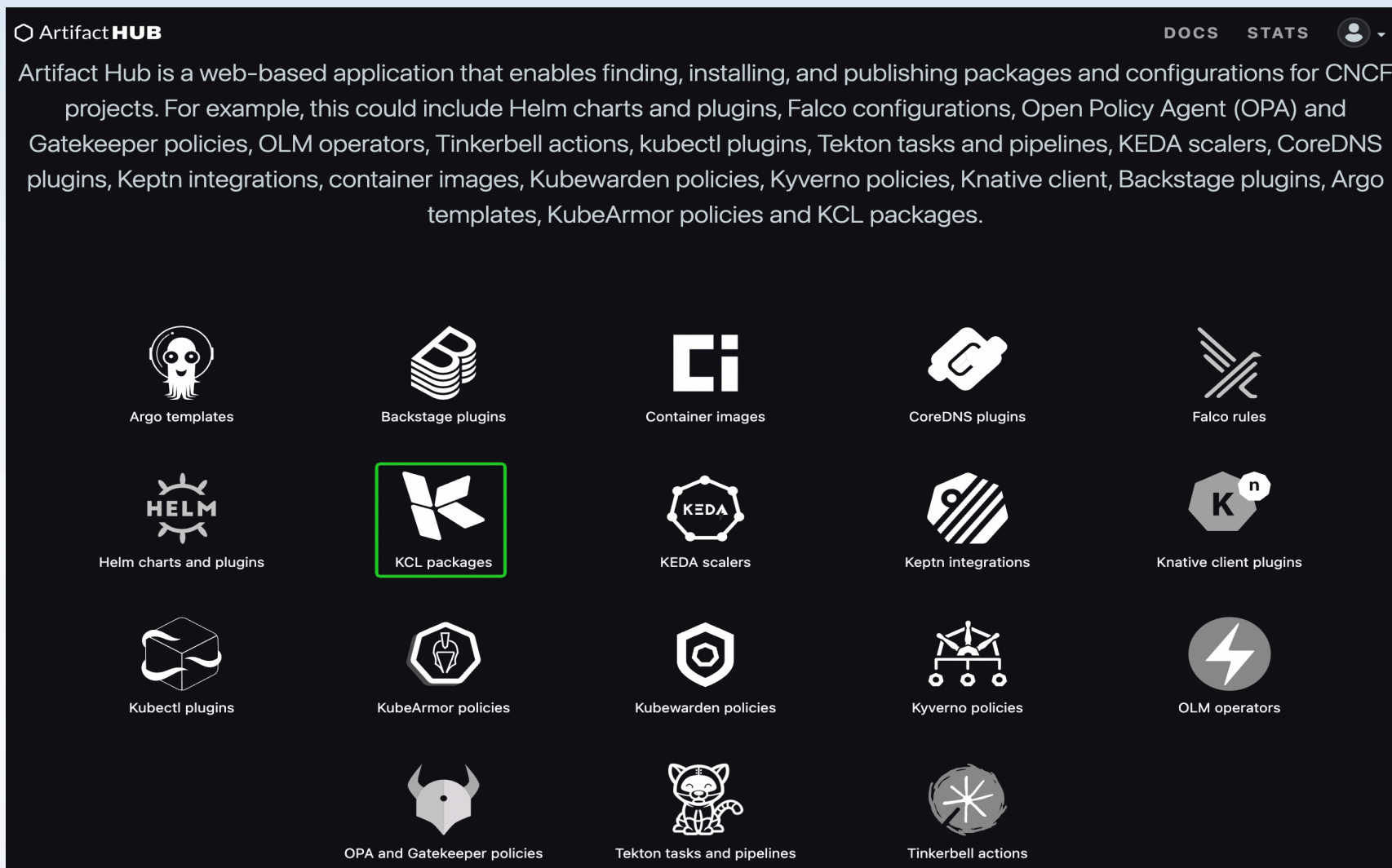
- **多语言 SDK:** Rust, Go, Python, Java SDK
- **包管理支持:** KPM 工具和多种 Registry /Hub支持
- **数据和 Schema 集成:** KCL Import/Export 工具
- **运行时集成:** 使用 KCL Operator 而不是重复开发 Kubernetes Admission Webhook
- **KRM 支持:** 统一的规范和插件支持 e.g., kubectl-kcl plugin, helm-kcl plugin, kustomize-kcl plugin, kpt-kcl-plugin ...
- **平台工程工具支持:** 作为 DCM 语言配合不同引擎/编排器进行应用交付

KCL 周边工具



Artifacthub & KCL

- **开箱即用**: 一行命令添加依赖
e.g., `kcl mod add k8s`, 现阶段官网模型 **200+** (欢迎共建)
- **多种场景支持**: 配置编辑、校验、模型抽象, Kubernetes 生态模型, Terraform 生态模型, 应用配置 ...
- **多种 Registry /Hub支持**:
Docker Hub, ghcr.io, ...



实践: Kubernetes 动态配置管理

- 少数几行 KCL 代码即可完成对应配置编辑功能 (客户端和运行时代码可以复用)
- 无需开发额外的 Kubernetes Webhook 编辑和验证配置

- Step 1. 在集群当中安装 KCL Operator
- Step 2. Apply KCL and K8s manifests

```
apiVersion: krm.kcl.dev/v1alpha1
kind: KCLRun
metadata:
  name: set-annotation
spec:
  params:
    annotations:
      managed-by: kcl-operator
  # Resource modification can be achieved with just one line of KCL code
  source: |
    items = [item | {metadata.annotations: option("params").annotations} for item in option("items")]
```

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  annotations:
    app: nginx
spec:
  containers:
    - name: nginx
      image: nginx:1.14.2
      ports:
        - containerPort: 80
```

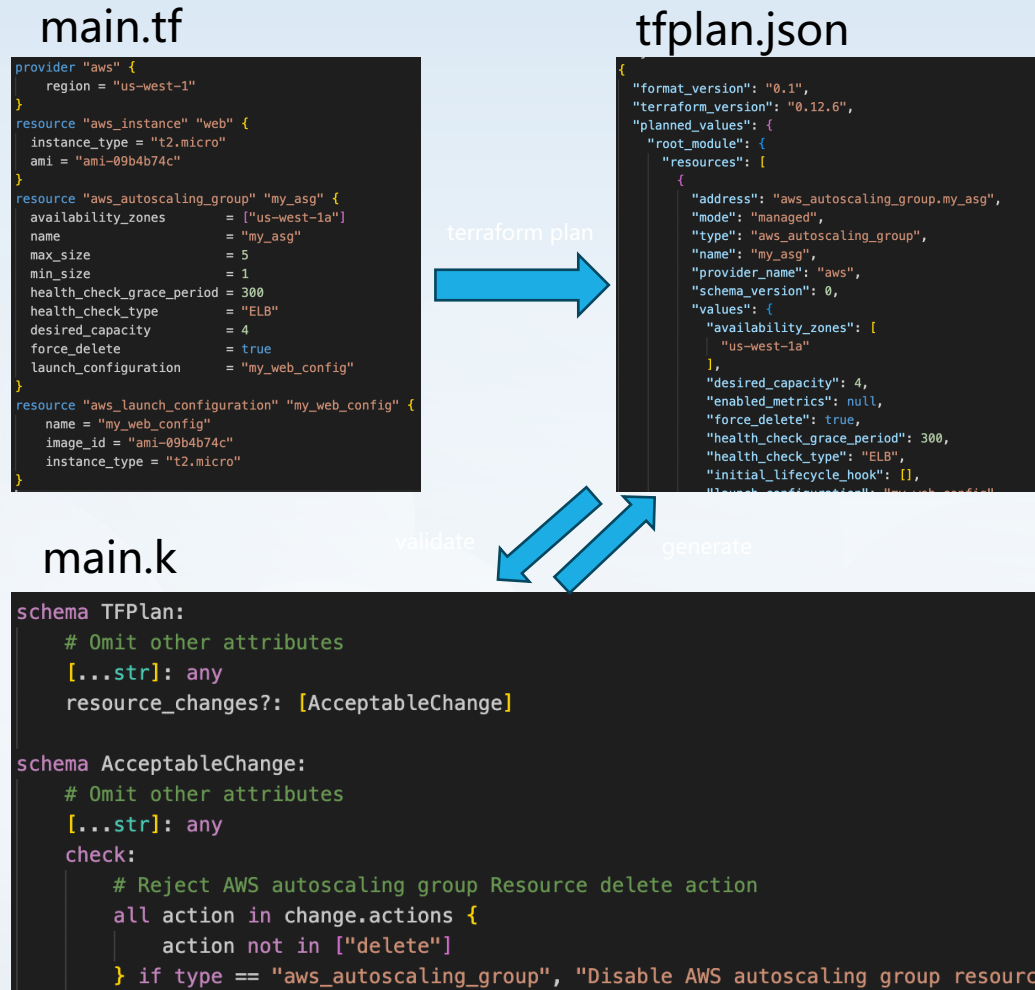
- Step 3. 获得资源 Mutation 结果

```
kubectl get po nginx -o yaml | grep kcl-operator

managed-by: kcl-operator
```

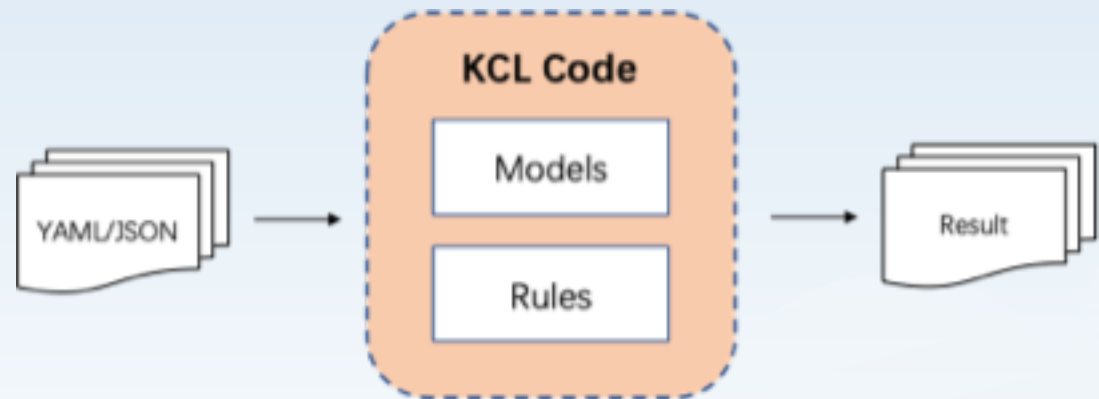
Reference the annotation modification model on OCI
source: oci://ghcr.io/kcl-lang/set-annotation

实践: Terraform 配置验证



kcl-vet tfplan.json main.k

Validation Process



- ✓ **多种数据支持**：JSON/YAML 等数据支持
- ✓ **结构定义**：Schema 结构化定义及自定义错误支持
- ✓ **生态集成**：OpenAPI/Terraform Provider Schema 转换 KCL Schema 支持
- ✓ **开箱即用**：丰富的配置策略模型库和代码示例 (欢迎共建)

https://kcl-lang.io/docs/user_docs/guides/working-with-terraform/validation

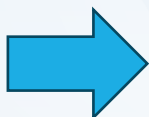
实践: Kubernetes 配置生成

Standalone KCL Form

```
import .app

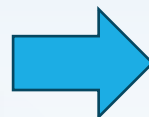
app.App {
  name = "app"
  containers.nginx = {
    image = "nginx"
    ports = [{containerPort = 80}]
  }
  service.ports = [{ port = 80 }]
}
```

UI/CLI/API



KRM KCL Form

```
apiVersion: krm.kcl.dev/v1alpha1
kind: KCLRun
metadata:
  name: web-service
  metadata:
    annotations:
      krm.kcl.dev/version: 0.0.1
      krm.kcl.dev/type: abstraction
      documentation: >=
        Web service application abstraction
spec:
  params:
    name: app
    containers:
      nginx:
        image: nginx
        ports:
          containerPort: 80
    labels:
      name: app
  source: oci://ghcr.io/kcl-lang/web-service
```



Kubernetes Manifests

```
apiVersion: v1
kind: Deployment
metadata:
  name: nginx
  namespace: ng
spec:
  replicas: 1
  selector:
    matchLabels:
      app.kubernetes.io/name: nginx-example
      app.kubernetes.io/env: dev
      app.kubernetes.io/instance: nginx-example-dev
      app.kubernetes.io/component: nginx-exampledev
  template:
    metadata:
      labels:
        app.kubernetes.io/name: nginx-example
        app.kubernetes.io/env: dev
        app.kubernetes.io/instance: nginx-example-dev
        app.kubernetes.io/component: nginx-exampledev
    spec:
      containers:
        - image: nginx:latest
          name: main
          ports:
            - containerPort: 80
              protocol: TCP
          resources:
            limits:
              cpu: 100m
              memory: 100Mi
              ephemeral-storage: 1Gi
            requests:
              cpu: 100m
              memory: 100Mi
              ephemeral-storage: 1Gi
```


在蚂蚁和其他公司的实践

1K/day

Pipelines

10K+/day

KCL Compilations

1 : 9

Plat : Dev

100K+

Commits

600+

Contributors

5.7K+

Projects

100K+

Pods

10K+

Workloads

Adopted by



欢迎加入我们

- Web Site

- <https://kusionstack.io/>
- <https://kcl-lang.io/>

- Github

- <https://github.com/KusionStack/>
- <https://github.com/kcl-lang>

- Slack

- KusionStack
- CNCF KCL Slack Channel

微信群小助手



钉钉群





Thanks.

